Module 2 Tangent: Error Handling

Error Models

The return of “differences across languages” (and APIs)

1.

Error Models

I'll reuse the title again, sue me.

Error Models

Definitions

An error model describes a style of handling errors for a function/system.

- Programming languages typically have a “preferred” error model.
- In practice, multiple models are used depending on the scenario.
  - Each error model has it’s own advantanges/disadvantages!

Tradeoffs

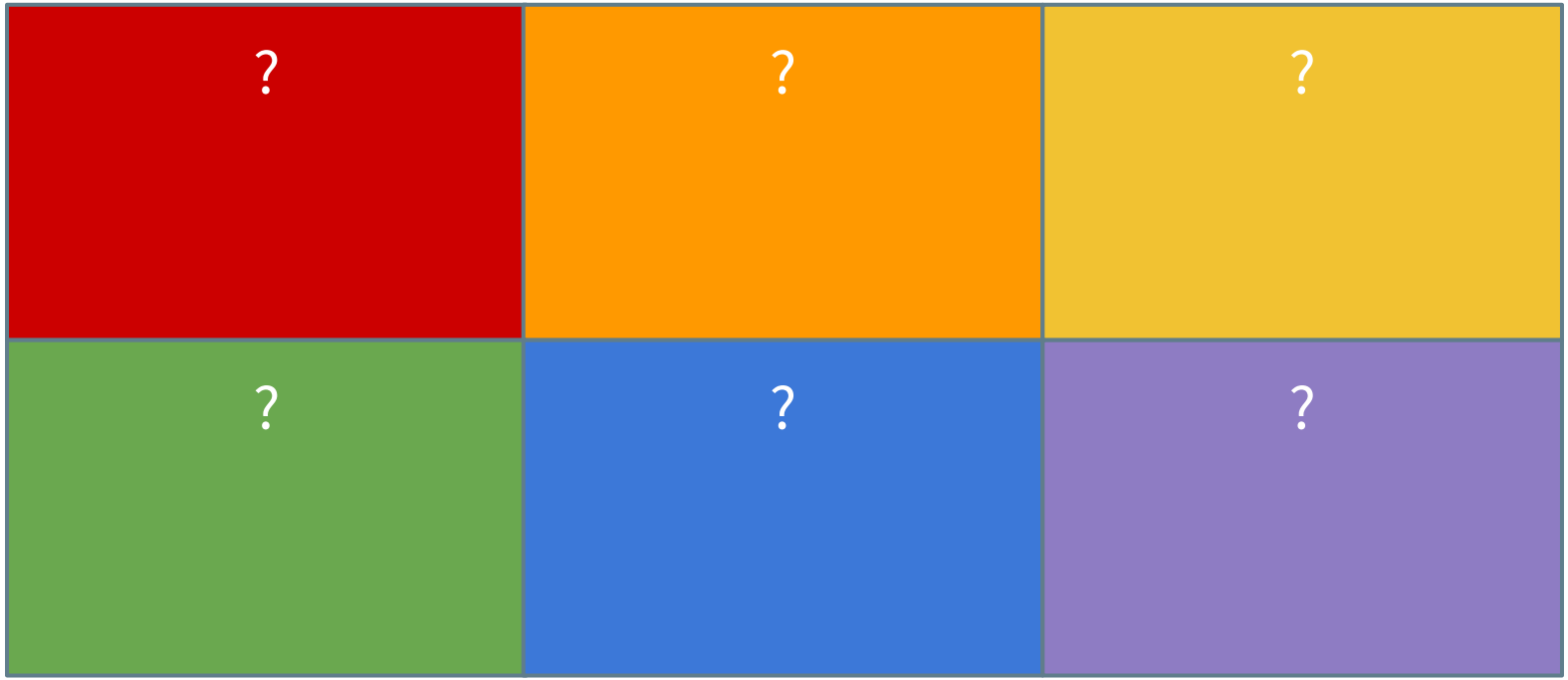
Each error model has their own tradeoffs. Some criteria for evaluating these:

- Safety: Is the error part of the type system, and/or will the language enforce handling
- Separation: To what extent is the error “separated” from normal operations?
  - Too much, and errors can be ignored.
  - Too little, and they’re hard to work with.
- Composition: How seamlessly can errors be composed with other parts of a program?

Suggested Reading:

Midori: The Error Model

~Joe Duffy (Pulumi, Microsoft)



Error Models

Sentinel Values	?	?
?	?	?

Error Models

```java
String string = …;

int index = string.indexOf("other");

if (index != -1) {                          //Sentinel Value: -1
  //success!                                //concern: error can be ignored!
} else {
  //not found
}

string.charAt(index);                       //potential error: charAt(-1)


Map<String, Object> map = …;
map.get("missing"); //null                  //concern: map could contain null
map.containsKey("missing"); //true
```

Sentinel Values: Java

Sentinel Values	Error Codes (and values)	?
?	?	?

Error Models

```
std::string string = …;

errno = 0;                                  //Reset errno to 0
char* end;                                  //concern: global state!
long result = strtol(
  string.c_str(), &end, 10);

if (errno == 0) {                           //Error Code: errno != 0
  //success!                                //concern: error can be ignored!
} else {                                    //advantage: can return any long
  //failure
}


//Alternative: return error code            //advantage: avoids global state
int result = main();                        //concern: cannot return a value
```

Error Codes: C++

```
std::string string = …;

int number;
switch(parseInt(&string, &number)) {      //concern: &number is… weird
  case SUCCESS:                           //concern: error can be ignored!
  //success!
  case OVERFLOW:
  case UNDERFLOW:
  case INCONVERTIBLE:
  //failure
}
```

Error Codes: C++

```
//Remember, enums are just integers! (SUCCESS = 0)
enum ParseIntError { SUCCESS, OVERFLOW, UNDERFLOW, INCONVERTIBLE };

ParseIntError parseInt(const std::string &string, int &out) {
  errno = 0;
  char* end;
  long result = strtol(string.c_str(), &end, 10);

  if ((errno == ERANGE && result == LONG_MAX) || result > INT_MAX) {
    return OVERFLOW;
  } else if ((errno == ERANGE && result == LONG_MIN) || result < INT_MIN) {
    return UNDERFLOW;
  } else if (string.front() == '\0' || *end != '\0') {
    return INCONVERTIBLE;
  }

  out = result;
  return SUCCESS;
}
```

`parseInt` implementation... yeowch.

Sentinel Values	Error Codes (and values)	Multiple Returns
?	?	?

Error Models

```go
var str string = …

number, err := strconv.ParseInt(str)      //Multiple Returns: (int, error
                                          //advantage: local, unique error
                                          //advantage: separation of error


if err != nil {                           //concern: error can be ignored
    //success!
} else {
    //failure
}
```

Multiple Returns: Go

Sentinel Values	Error Codes (and values)	Multiple Returns
Sum Types	?	?

Error Models

```
let string = …

let return = parseInt(string)

match return with
    | Int num -> //success!
    | Error err -> //failure

type ParseIntReturn =                          //advantage: enumeration of
    | Int of Int32                             //all possible return types
    | Error of String
```

Sum Types: F#

Sentinel Values	Error Codes (and values)	Multiple Returns
Sum Types	Result Types (and Monads)	?

Error Models

Result Types

Definition

A Result type represents an operation that may either be a success or failure.

- Result.success() (or similar) contains the “actual” result of the function.
- Result.failure() (or similar) contains the error result of the function.
- Result often has utilities for accessing values and composing Results.
  - Result.map(), Result.orElse(), etc.
- We’ve seen one already: Optional<T>

Monads!

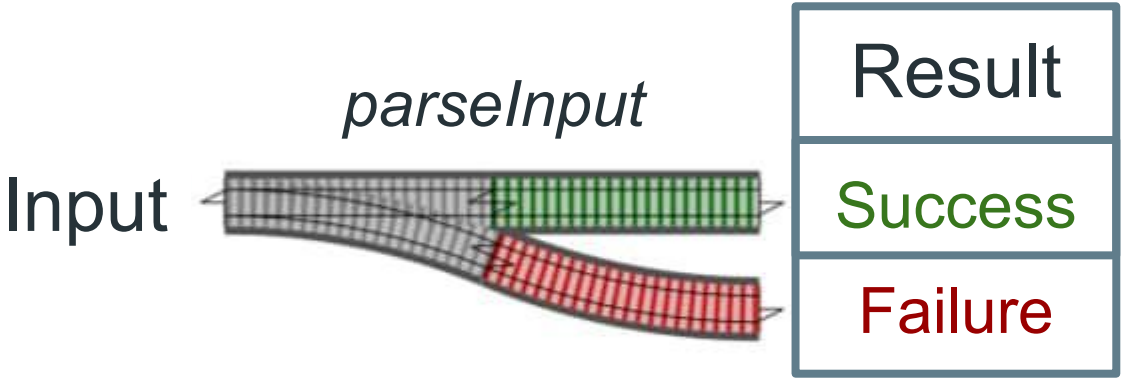
~~A monad is a monoid in the category of endofunctors, what’s the problem?~~

- A Monad is a type designed for composing related operations together.
  - IO Monad composes IO side effects
  - Maybe Monad composes optional values
  - A key operation, **bind**, allows converting non-monad functions to monad versions.
- Result is a monad (as is Optional<T>)
- JavaScript Promises are *basically* a Monad!

Railway Oriented Programming (link)

Idea: Chain Success/Failure Operations

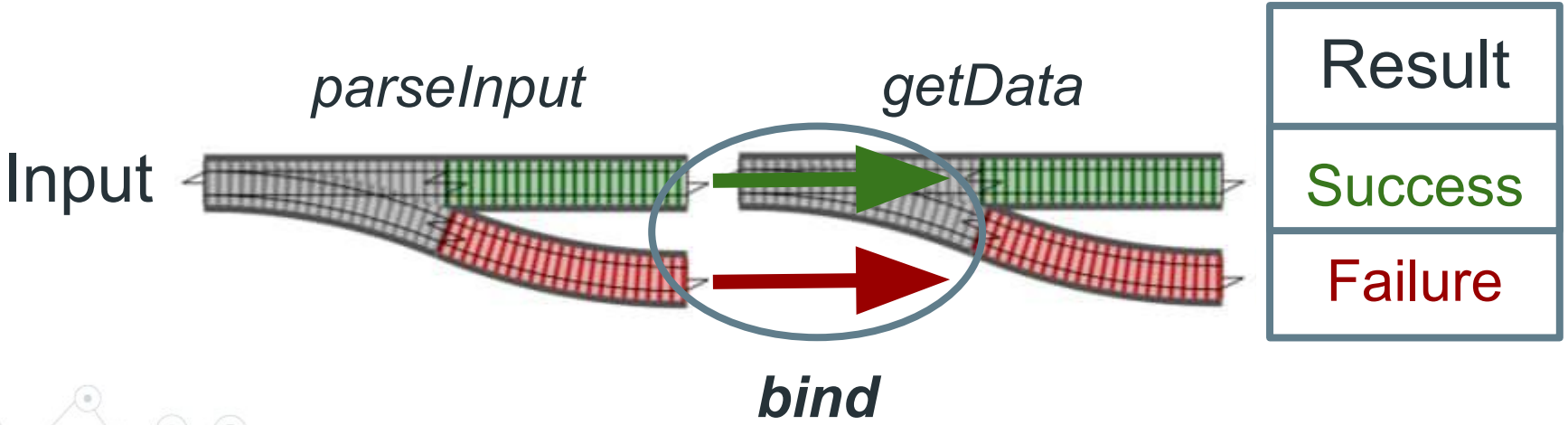
Given a function that can return a success or failure Result…



Railway Oriented Programming (link)

Idea: Chain Success/Failure Operations

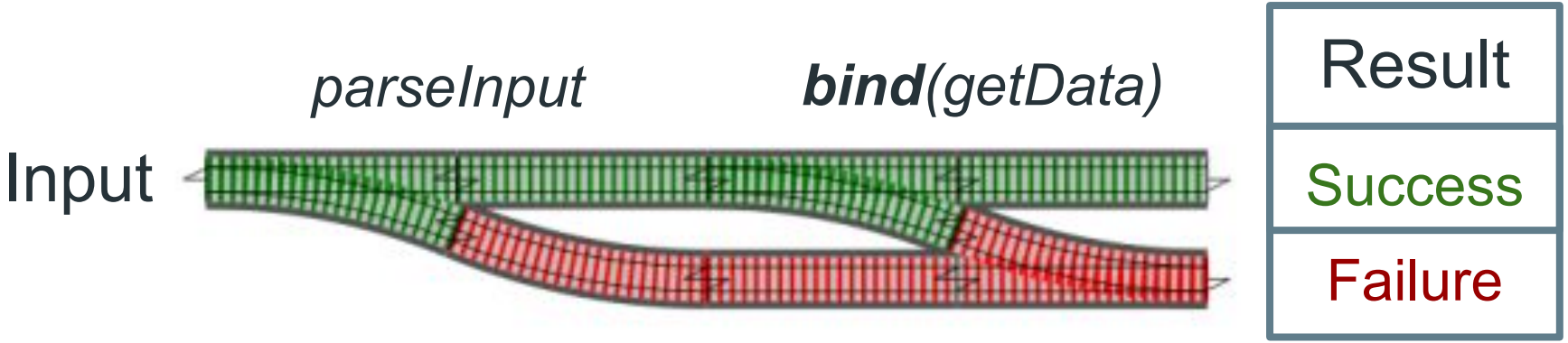
…we can combine it with other functions by connecting success/failure tracks…



Railway Oriented Programming (link)

Idea: Chain Success/Failure Operations

…to create a new function that chains their behavior and returns a Result.



```swift
let string = …

let result: Result<Int, Error>
  = parseInt(string)
if let number = result {                          //advantage: must be checked
    //success!
} else {
    //failure
}


                                                  //language syntax support:
let number = result ?? -1                         //default value on Failure
let result: Int? = try? parseInt(string)          //convert Result to Optional
let number: Int = try! parseInt(string)           //assert Result is Success
```

Result Types: Swift

Sentinel Values	Error Codes (and values)	Multiple Returns
Sum Types	Result Types (and Monads)	Exceptions (checked/unchecked)

Error Models

Exceptions

Definition

An exception is a method of error handling that interrupts control flow.

- Exceptions are thrown (raised) and “travel” up the call stack until caught.
  - Roughly like a second return value.
- Exceptions are generally expensive compared to normal control flow.
  - Must lookup catch/finally blocks.
  - Often creates a stack trace (debug).

Checked & Unchecked

Java is the only major language to support checked exceptions for static validation.

- Checked exceptions are part of the function declaration and **must** be caught/rethrown.
  - Best for errors that are recoverable.
  - May require updating multiple functions.
  - Java: Integrates poorly with lambdas.
- Unchecked exceptions do not need to be declared and can be thrown from anywhere.
  - Best for programming errors (bugs).

```java
String string = …;

try {
  int n = Integer.parseInt(string);
  //success!
} catch (NumberFormatException e) {
  //failure
}

//concern: this compiles???
Integer.parseInt(string);
//java.lang.NumberFormatException
```

```java
String string = …;

try {
  var tokens = Lexer.lex(string);
  //success!
} catch (LexException e) {
  //failure
}

//advantage: this errors!
Lexer.lex(string);
//(no runtime error - checked!)

//concern: painful with lambdas
Stream.of(string).map(Lexer::lex);
```

Java: Unchecked vs Checked Exceptions

How do Checked Exceptions compare to Result Types?

Largely a question of language features, but Result Types are less effective for Void functions.

Sentinel Values	Error Codes (and values)	Multiple Returns
Sum Types	Result Types (and Monads)	Exceptions (checked/unchecked)

Error Models

2.

Throw... or... no!!!

The hot new game show that no one has heard of!

*For each scenario, what do you think the best (*ish) error approach is?*

Throw: Checked / Unchecked Exceptions
No Throw: Sentinel Values / Result Types

Write a String to a File.

Survey Says: ***Throw*** *(Checked Exception)*

Find the index of the largest number in an array.

Survey Says: ***No Throw*** *(Result Type)*

Looking up an entry in a Map.

Survey Says: ***No Throw*** *(Result Type)*

Passing an argument that doesn't meet preconditions.

Survey Says: ***Throw*** *(Unchecked Exception)*

Diving a number by zero.

Survey Says: ***Throw*** *(Unchecked Exception)*
(Sentinel Value: IEEE-754 floating point)

Adding a duplicate entry to a global registry.

Survey Says: ***Throw*** *(Checked Exception)*

Casting an Object to a String.

Survey Says: ***Throw*** *(Unchecked Exception)*
(Result Type could be reasonable too)

File IO failure loading system configuration file.

Survey Says: ***Throw*** *(Unchecked Exception).*

Module 2 Tangent: Error Handling

Error Handling

Not to be confused with “error ignoring”

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals